

QuantSparse3D 2026-08-27 Bi-weekly

Integrated Systems Laboratory (ETH Zürich)

Bowen Wang
Chi Zhang

bowwang@iis.ee.ethz.ch
chizhang@iis.ee.ethz.ch

Luca Benini

lbenini@iis.ee.ethz.ch

PULP Platform

Open Source Hardware, the way it should be!

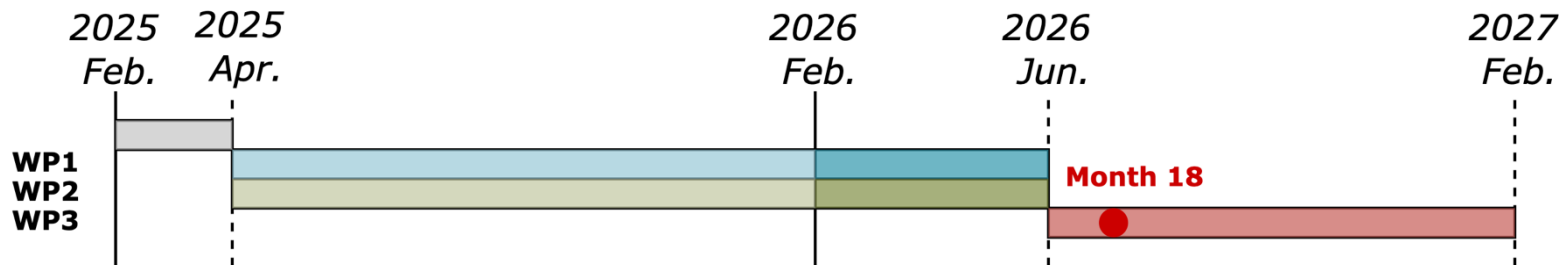


@pulp_platform 

pulp-platform.org 

youtube.com/pulp_platform 

Agenda



- **Progress Update**
 - Sparsity handling in memory endpoints
- **Timeline**
- **Discussion**

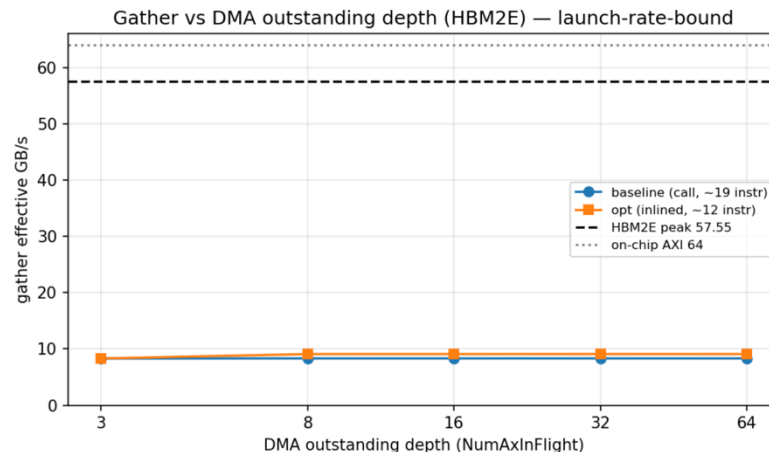
Recap: Core-issued Gather



- Core-issued gather, with an optimized kernel
 - each row offloaded *inline* via `dmsrc/dmdst/dmcpyi`
 - 12 instructions/iteration, 4 beats to transfer 64B burst
→ ideally **33%** bandwidth utilization
- Result:
 - 9.08 GB/s** → **16%** of peak (28.2 cyc/read)
 - 3.8× below the ideal 60.2%**
- Interpretation:
 - Core issuing request too slow? → **No**
The loop runs ahead of the DMA, fills the request FIFO,
then the core simply back-pressured on `dmcpyi`, or
spinning in `snrt_dma_wait_all`

`dma_issue_inner:`

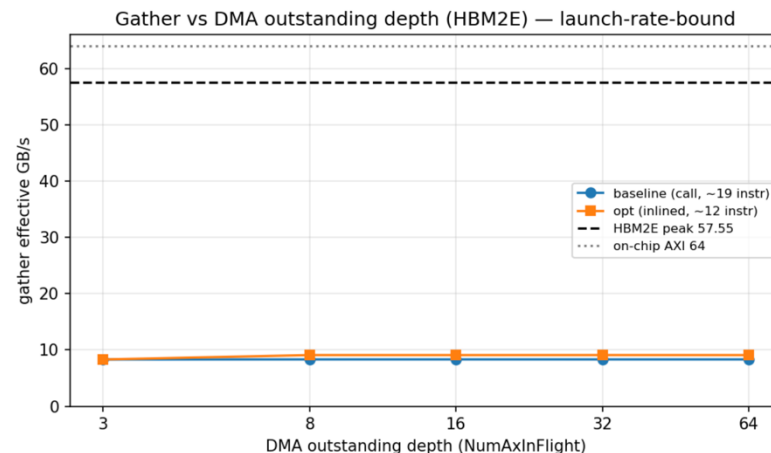
```
    addi a0, a5, 0      # a0 = d
    add  a1, a7, a6      # a1 = &index[i]
    lw   a1, 0(a1)      # a1 = index[i]
    A    addi a5, a5, 256 # d += row_bytes
    addi a6, a6, 4       # i += 1 (in bytes)
    slli a1, a1, 8       # a1 = index[i] * 256
    add  a2, a1, t0      # a2 = matrix + off
    dmsrc a2, a3         # DMA.src = a2 (a3=hi=0)
    li   a1, 0          # a1 = 0
    B    dmdst a0, a1    # DMA.dst = a0 (hi=0)
    dmcpyi a0, a4, 0     # launch: len=a4=256
    bne  a6, s1, b00     # loop while i != G
```



- ```

dma_issue_inner:
 ↑ addi a0, a5, 0 # a0 = d
 add a1, a7, a6 # a1 = &index[i]
 lw a1, 0(a1) # a1 = index[i]
 A addi a5, a5, 256 # d += row_bytes
 addi a6, a6, 4 # i += 1 (in bytes)
 slli a1, a1, 8 # a1 = index[i] * 256
 ↓ add a2, a1, t0 # a2 = matrix + off
 ↑ dmsrc a2, a3 # DMA.src = a2 (a3=hi=0)
 li a1, 0 # a1 = 0
 B dmdst a0, a1 # DMA.dst = a0 (hi=0)
 ↓ dmcpyi a0, a4, 0 # launch: len=a4=256
 bne a6, s1, b00 # loop while i != G

```



# Core-issued Gather



- Extending allowed in-flight AXI requests:
  - Add from 4  $\rightarrow$  32
  - Effective, achieved **29.2%** bandwidth utilization (max. achievable 30.8%)
  - The performance is core-side instruction loop bound
- **Solution: sparse-capable DMA engine**
  - Core configures the sparse transaction **once**
  - DMA engine **fetches the index stream** and generate disaggregated AXI requests
  - Additional semantics to cover:
    - Address pointer to index stream in L1
    - Per-index bit-width
    - Index format (offset, bitmap...)

`dma_issue_inner:`

```
 addi a0, a5, 0
 add a1, a7, a6
 lw a1, 0(a1)
 A addi a5, a5, 256
 addi a6, a6, 4
 slli a1, a1, 8
 add a2, a1, t0
 dmsrc a2, a3
 li a1, 0
 B dmdst a0, a1
 dmcpyi a0, a4, 0
 bne a6, s1, b00
```

```
a0 = d
a1 = &index[i]
a1 = index[i]
d += row_bytes
i += 1 (in bytes)
a1 = index[i] * 256
a2 = matrix + off
DMA.src = a2 (a3=hi=0)
a1 = 0
DMA.dst = a0 (hi=0)
launch: len=a4=256
loop while i != G
```

# Existing DMA ISA extensions

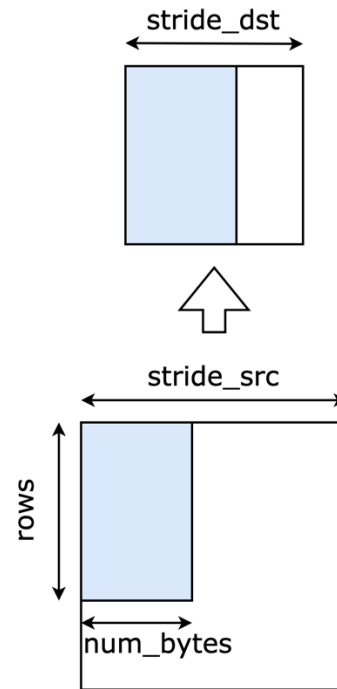


- 1D request

```
DMSRC src_base_addr
DMDST dst_base_addr
// issue; cfg = {direction, decouple_rw}
DMCPY num_bytes, cfg
```

- 2D request

```
DMSRC src_base_addr
DMDST dst_base_addr
// optional
DMSTR stride_src, stride_dst
// number of 1D transfer to perform
DMREP rows
// issue; cfg = {direction, is_twod,
// decouple_rw}
DMCPY num_bytes, cfg
```

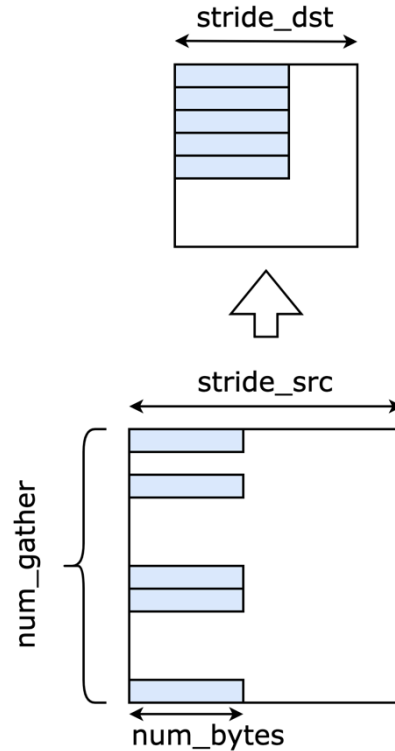


# Proposed Sparse DMA ISA extensions



- Only one additional for index format configuration
  - For L1-reside index address and formats
  - Reuse **DMREP** to specify the number of gathered targets
  - Additional issue flag to distinguish with other requests

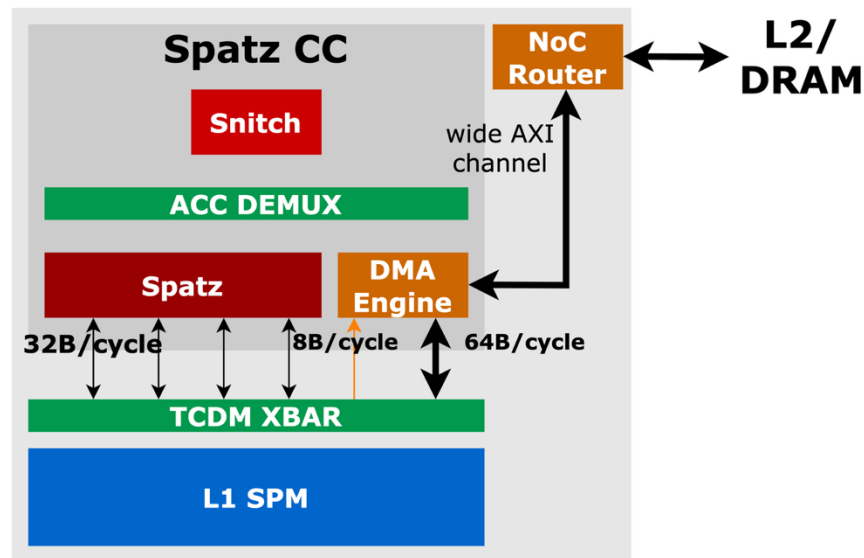
```
DMSRC src_base_addr
DMDST dst_base_addr
// index location;
 cfg: width/format
DMIDX idx_addr, {width, fmt}
// optional
DMSTR stride_src, stride_dst
// number of gather to perform
DMREP num_gather
// issue; cfg = {direction, is_gather,
 decouple_rw}
DMCPY num_bytes, cfg
```



# Sparse DMA Microarchitecture



- *Idea:* TCDM path for DMA engine for index fetching
  - **Frontend**
    - New instruction decoding;
    - Configuration capture
  - **Midend:**
    - Fetch/pre-fetch index stream w/ control logic;
    - Address generation with indices;
    - AXI requests generation
  - *Parallel midend by 2D DMA engine*





# Sparse DMA Microarchitecture



- Implementation status
  - **Frontend:**
    - New instruction added, decoder in frontend done;
    - Verified sparse configurations captured by frontend;
  - **Midend:**
    - Standalone implementation finished
- *Next step:*
  - Integrate the midend to Spatz DMA engine for functional verification
  - Gather kernel implementation with the new DMA instructions
  - Performance benchmark

